ISIL

PROCESO DE APRENDIZAJE 2

ALGORITMO Y ESTRUCTURA DE DATOS BASADOS EN INTELIGENCIA ARTIFICIAL

NRC: 4134 / 2026-20

Profesor: WILDER JULIO ESPINOZA BRAVO

GRUPO 4

- JACK STEFANO TELLO CABALLERO
- JOHANN JOASH ISHAQ CONDOR VARGAS
- CARLOS JOSUE FLORES PAREJA
- JEAN LUC MELLET DEL CASTILLO
- MARCELO GABRIEL GARCÍA CAMPOBLANCO

2. Introducción y Análisis del Caso

- Descripción del problema de la organización y la naturaleza dinámica de las solicitudes.
- Justificación del uso de estructuras dinámicas (listas) frente a estructuras estáticas (arreglos), destacando que las listas enlazadas no requieren memoria contigua.
- Breve mención sobre cómo el equipo utilizó herramientas de IA de forma crítica y académicamente responsable para auditar o generar código durante el desarrollo

En este trabajo nos tocó resolver un problema clásico donde una organización necesita manejar una lista de solicitudes de atención que cambia a cada rato: entran nuevos casos, otros se borran, hay que reubicar algunos o revisarlos en bucle.

Para esto, decidimos usar **estructuras de datos dinámicas (listas enlazadas)** en lugar de arreglos estáticos tradicionales. La gran ventaja de usar listas es que no necesitamos reservar un bloque de memoria contigua desde el inicio, lo cual nos salva la vida cuando los datos crecen o disminuyen de tamaño de forma impredecible en plena ejecución.

Nota sobre IA: Durante el desarrollo del proyecto, el equipo utilizó herramientas de Inteligencia Artificial de forma crítica y responsable, principalmente para revisar la sintaxis de nuestros códigos en Java, auditar posibles errores de lógica en los punteros (nullPointerException) y ordenar las ideas principales de la documentación.

.

3. Actividad 1: Fundamentos y representación de listas enlazadas *(Responsable: Johann)*

- **Definición de Nodo:** Explicar que es la unidad estructural básica que contiene un dato y una referencia al siguiente nodo.
- **Definición de Lista Simple:** Detallar que es una estructura lineal de recorrido unidireccional donde cada nodo tiene un único puntero (next) y el último apunta a null para indicar el fin.
- **Representación gráfica:** Diagrama con al menos 5 nodos aplicados al caso de las solicitudes.

Definición de Nodo: Es la unidad básica de almacenamiento. Cada nodo guarda una porción de información (en nuestro caso, los datos de la solicitud como el ID, el nombre del solicitante, el tipo y el estado) y un puntero o referencia (siguiente) que apunta al siguiente nodo de la cadena.

Definición de Lista Simple: Es una estructura lineal donde el recorrido es de un solo sentido (unidireccional). Cada elemento se conecta únicamente con el que le sigue, y el último nodo de todos apunta a null para indicarnos que ahí se acaba la lista.

Representación Gráfica (Aplicada al caso con 5 nodos):

[ID: 101 | Ana] -> [ID: 102 | Luis] -> [ID: 103 | María] -> [ID: 104 | Carlos] -> [ID: 105 | Sofía] -> nullpa



4. Actividad 2: Recorrido e inserción *(Responsable: Jack)*

- **Diseño de algoritmos:** Código o pseudocódigo de las 4 operaciones solicitadas.
- **Explicación de cambios y complejidad:**
    - Inserción al inicio: Explicar el cambio de punteros y su complejidad .
    - Inserción al final: Explicar el recorrido hasta el último nodo y su complejidad .
    - Inserción en posición específica: Explicar la reconexión en el medio de la lista y su complejidad .

Para esta parte, implementamos los métodos básicos en Java para movernos por la estructura y agregar elementos:

- **Inserción al inicio:** Creamos el nuevo nodo y hacemos que su puntero siguiente apunte a la cabeza actual, cambiando luego la cabeza hacia el nuevo nodo. Su complejidad temporal es de porque se hace al toque, sin importar cuántos elementos haya.
- **Inserción al final:** Recorremos toda la lista desde la cabeza hasta encontrar el nodo cuyo siguiente sea null, y ahí enganchamos el nuevo elemento. Su complejidad es porque crece conforme la lista se hace más larga.
- **Inserción después de un nodo específico:** Buscamos un nodo por su ID y, al encontrarlo, reacomodamos los punteros para meter el nuevo nodo en el medio sin romper la secuencia. Su complejidad también es por la búsqueda previa.

5. Actividad 3: Lista enlazada simple (Búsqueda, modificación y eliminación)
(Responsable: Jean Luc)

- **Diseño de algoritmos:** Código para buscar, modificar y eliminar un nodo según criterios del caso.
- **Reajuste de enlaces:** Explicación técnica de cómo se evita que la lista se rompa al eliminar un nodo.
- **Análisis de eficiencia:** Comparación teórica de la inserción vs. eliminación dependiendo de si ocurre al inicio, medio o final de la estructura.

Basándonos en el código implementado, usamos la clase Nodo con atributos detallados (idSolicitud, solicitante, tipoAtencion, estado) y una clase gestora (ListaSolicitudes):

- **Búsqueda por ID:** Recorre la lista comparando el ID buscado con el de cada nodo hasta hallarlo o llegar al final (null).

- **Modificación:** Utiliza la función de búsqueda para ubicar el nodo y actualizar directamente sus campos de estado o tipo de atención de forma rápida.
- **Eliminación y reajuste de enlaces:** Si borramos el primer elemento, solo movemos la cabeza al siguiente. Si está en el medio o al final, usamos un puntero anterior y uno actual para saltarnos el nodo eliminado (anterior.siguiente = actual.siguiente), evitando así que la lista se rompa o perdamos el rastro de los demás elementos.
- **Análisis de eficiencia:** Insertar o eliminar al inicio es súper eficiente (), mientras que hacerlo en posiciones intermedias o al final requiere recorrer los elementos, resultando en un costo de .

6. Actividad 4: Listas dobles y circulares *(Responsable: Marcelo)*

- **Caso de uso - Lista Doble:** Situación del problema donde sea obligatorio navegar en ambos sentidos. Incluir justificación técnica y diagrama.
- **Caso de uso - Lista Circular:** Situación donde se requiera un ciclo continuo de revisión sin un nodo final que apunte a null. Incluir justificación técnica y diagrama.

Caso de uso - Lista Doble: Ideal para el historial de atención donde el operador necesita navegar tanto hacia adelante como hacia atrás entre los casos de los clientes. Se justifica porque cada nodo tiene dos punteros (siguiente y anterior), facilitando el retroceso sin volver al inicio.

Caso de uso - Lista Circular: Útil cuando el sistema necesita dar rondas de revisión continua a las solicitudes pendientes (un bucle infinito controlado de atención). Se justifica porque el último nodo no apunta a null, sino que se conecta de vuelta con la cabeza, manteniendo el ciclo activo.

7. Conclusiones *(Responsable: Carlos / Equipo)*

- Reflexión final sobre la eficiencia de los algoritmos desarrollados y el manejo de tipos de datos dinámicos mutables frente a los estáticos.

Haciendo este trabajo nos dimos cuenta de que, aunque al principio manejar punteros y acordarse de actualizar los null puede confundir un poco, las estructuras dinámicas como las listas enlazadas son súper útiles en la vida real. A diferencia de los arreglos fijos, nos dan la libertad de manipular los datos de manera flexible según lo que pida el sistema, optimizando el uso de memoria y adaptándose recontra bien a los cambios constantes de una empresa. ¡Fue un reto, pero chambeando en equipo salió adelante!